INDEX

NAME: VIKASHINI M STD.: _____ SEC.: _____ ROLL NO.: 241801314 SUB.: Computer Network

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
1.	16/7/26	Study of various network commans in Linux & Windows		
3.	23/7/26	Cisco Packet Tracer & Network Devices Application		
2.	27/7/26	Study of network cables		
4.	30/7/26	Setup & configure LAN using Switch & Ethernet cable		
5.	1/8/26	Experiments on packet trace tool		
6.	6/8/26	Error deduction & correction using Hamming Code		
7	8/8/26	Flow control at data link layer using Sliding window protocol		
8a	13/8/26	Live Host discovery using NMAP scans		
8b	20/8/26	Wireless LAN configuration		
9	29/8/26	Implementation of Subnetting		
10a	10/9/26	Internetworking with Router		

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
10 b	12/9/26	Internetworking with wireless router, DHCP servers and Internet cloud		
11 a	17/9/26	Static Routing protocol configuration		
11 b	24/9/26	Simulate RIP using CISCO packet Tracer		
12 a	26/9/26	ECHO client Server using TCP Sockets		
12 b	1/10/26	Chat client-Server using TCP sockets		
13	8/10/26	Implementing ping program		
14	10/10/26	Packet sniffing using Raw Sockets		
15	10/10/26	Web log analysis using Webalizer		

Completed

Exp 1: Stu

Date: 16/7/26

WINDOWS

1. arp - A
2. hostna
3. ipcon
4. nbts
5. net
6. nsl
7. pat
8. pi
9. ro

1. arp

2.

Teacher's Sign / Remarks

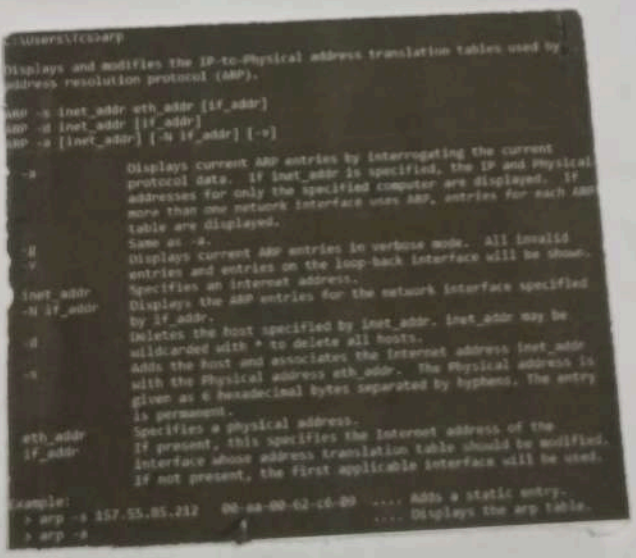
Exp 1: Study of various network commands used in Linux and windows

Date: 16/9/24

WINDOWS COMMANDS

1. arp - Address Resolution protocol
2. hostname - Computer name
3. ipconfig - Computer Requirements
4. nbtstat - Network bios
5. netstat - Network statistics
6. nslookup - Name Server
7. pathping - Combination of ping & tracert commands
8. ping - Package Internet dropper
9. route - Manipulate the IP routing table

1. arp



2. hostname



3. ipconfig

```
C:\Users\Tcs>ipconfig

Windows IP Configuration


Ethernet adapter Ethernet:

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::90bf:3a79:be3d:18b0%14
   IPv4 Address. . . . . . . . . . . : 172.16.10.113
   Subnet Mask . . . . . . . . . . . : 255.255.252.0
   Default Gateway . . . . . . . . . : 172.16.8.1
```

4. nbstat

```
C:\Users\Tcs>nbtstat

Displays protocol statistics and current TCP/IP connections using NBT
(NetBIOS over TCP/IP).

NBTSTAT [ [-a RemoteName] [-A IP address] [-c] [-n]
        [-r] [-R] [-RR] [-s] [-S] [interval] ]

  -a   (adapter status) Lists the remote machine's name table given its name
  -A   (Adapter status) Lists the remote machine's name table given its
                        IP address.
  -c   (cache)          Lists NBT's cache of remote [machine] names and their IP addresses
  -n   (names)          Lists local NetBIOS names.
  -r   (resolved)       Lists names resolved by broadcast and via WINS
  -R   (Reload)         Purges and reloads the remote cache name table
  -S   (Sessions)       Lists sessions table with the destination IP addresses
  -s   (sessions)       Lists sessions table converting destination IP
                        addresses to computer NETBIOS names.
  -RR  (ReleaseRefresh) Sends Name Release packets to WINS and then, starts Refresh

  RemoteName   Remote host machine name.
  IP address   Dotted decimal representation of the IP address.
  interval     Redisplays selected statistics, pausing interval seconds
               between each display. Press Ctrl+C to stop redisplaying
               statistics.
```

5. netstat

```
C:\Users\Tcs>netstat

Active Connections

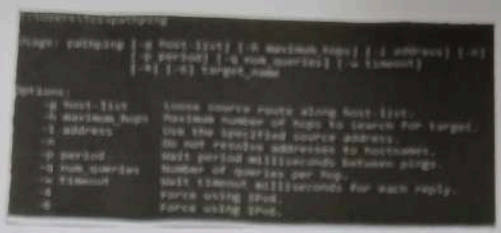
  Proto  Local Address          Foreign Address        State
  TCP    172.16.10.113:50979    172.16.8.1:domain      ESTABLISHED
  TCP    172.16.10.113:60031    4.213.25.242:https     ESTABLISHED
  TCP    172.16.10.113:61835    sf-in-f188:5228        ESTABLISHED
  TCP    172.16.10.113:61909    4.213.25.240:https     ESTABLISHED
  TCP    172.16.10.113:61915    123:http               TIME_WAIT
  TCP    172.16.10.113:61917    lcmaaa-aw-in-f14:https ESTABLISHED
  TCP    172.16.10.113:61918    lcmaaa-aw-in-f14:https TIME_WAIT
  TCP    172.16.10.113:61919    142.251.150.4:https    ESTABLISHED
  TCP    172.16.10.113:61920    142.251.150.4:https    ESTABLISHED
  TCP    172.16.10.113:61921    20.42.65.88:https      TIME_WAIT
  TCP    172.16.10.113:61922    cgk02s06-in-f6:https   ESTABLISHED
  TCP    172.16.10.113:61923    172.217.113.4:https    ESTABLISHED
  TCP    172.16.10.113:61924    123:http               TIME_WAIT
  TCP    172.16.10.113:61925    104.18.32.47:https     ESTABLISHED
  TCP    172.16.10.113:61926    172.217.117.4:https    ESTABLISHED
  TCP    172.16.10.113:61927    150.171.110.85:https   ESTABLISHED
  TCP    172.16.10.113:61928    lcbomp-in-f84:https    ESTABLISHED
  TCP    172.16.10.113:61929    lcbomp-in-f84:https    ESTABLISHED
  TCP    172.16.10.113:61930    lcbomp-in-f84:https    ESTABLISHED
  TCP    172.16.10.113:61933    1:https                ESTABLISHED
  TCP    172.16.10.113:61935    a23-202-229-22:https   ESTABLISHED
  TCP    172.16.10.113:61936    57.155.48.216:https    ESTABLISHED
  TCP    172.16.10.113:61937    123:http               ESTABLISHED
  TCP    [fe80::90bf:3a79:be3d:18b0%14]:1521  DESKTOP-G0LVE4J:49674  ESTABLISHED
  TCP    [fe80::90bf:3a79:be3d:18b0%14]:49674  DESKTOP-G0LVE4J:1521   ESTABLISHED
```

6. nslookup

```
C:\Users\Tcs>nslookup google.com
Server:  UnKnown
Address:  172.16.8.1

Non-authoritative answer:
Name:    google.com
Addresses:  2404:6800:4007:808::200e
          142.250.77.110
```

7. pathping

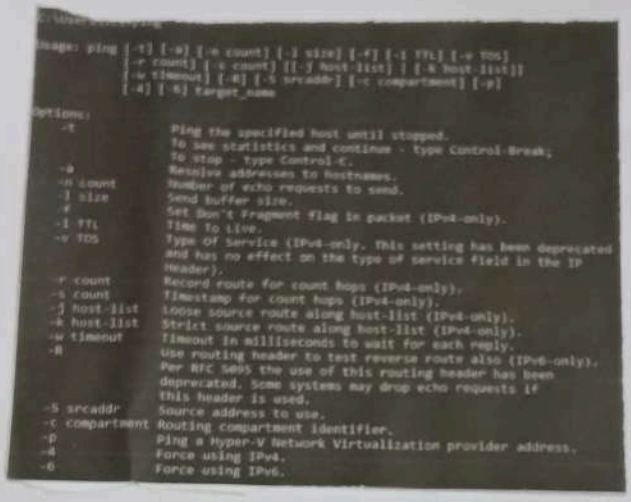


```
C:\Users\Tca>pathping

Usage: pathping [-g host-list] [-h maximum_hops] [-i address] [-n]
                [-p period] [-q num_queries] [-w timeout]
                [-4] [-6] target_name

Options:
  -g host-list     Loose source route along host-list.
  -h maximum_hops  Maximum number of hops to search for target.
  -i address       Use the specified source address.
  -n               Do not resolve addresses to hostnames.
  -p period        Wait period milliseconds between pings.
  -q num_queries   Number of queries per hop.
  -w timeout       Wait timeout milliseconds for each reply.
  -4               Force using IPv4.
  -6               Force using IPv6.
```

8. ping

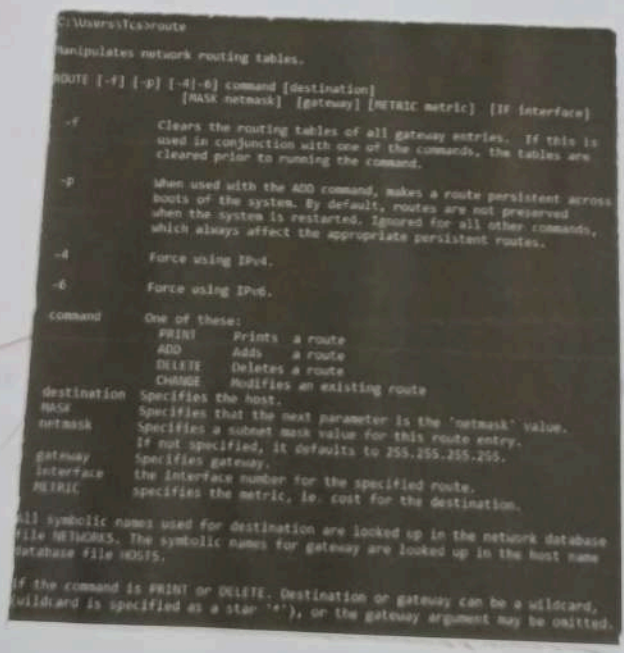


```
C:\Users\Tca>ping

Usage: ping [-t] [-a] [-n count] [-l size] [-f] [-i TTL] [-v TOS]
            [-r count] [-s count] [[-j host-list] | [-k host-list]]
            [-w timeout] [-R] [-S srcaddr] [-c compartment] [-p]
            [-4] [-6] target_name

Options:
    -t             Ping the specified host until stopped.
                   To see statistics and continue - type Control-Break;
                   To stop - type Control-C.
    -a             Resolve addresses to hostnames.
    -n count       Number of echo requests to send.
    -l size        Send buffer size.
    -f             Set Don't Fragment flag in packet (IPv4-only).
    -i TTL         Time To Live.
    -v TOS         Type Of Service (IPv4-only. This setting has been deprecated
                   and has no effect on the type of service field in the IP
                   Header).
    -r count       Record route for count hops (IPv4-only).
    -s count       Timestamp for count hops (IPv4-only).
    -j host-list   Loose source route along host-list (IPv4-only).
    -k host-list   Strict source route along host-list (IPv4-only).
    -w timeout     Timeout in milliseconds to wait for each reply.
    -R             Use routing header to test reverse route also (IPv6-only).
                   Per RFC 5095 the use of this routing header has been
                   deprecated. Some systems may drop echo requests if
                   this header is used.
    -S srcaddr     Source address to use.
    -c compartment Routing compartment identifier.
    -p             Ping a Hyper-V Network Virtualization provider address.
    -4             Force using IPv4.
    -6             Force using IPv6.
```

9. route



```
C:\Users\Tca>route

Manipulates network routing tables.

ROUTE [-f] [-p] [-4|-6] command [destination]
                  [MASK netmask]  [gateway] [METRIC metric]  [IF interface]

    -f           Clears the routing tables of all gateway entries.  If this is
                 used in conjunction with one of the commands, the tables are
                 cleared prior to running the command.

    -p           When used with the ADD command, makes a route persistent across
                 boots of the system. By default, routes are not preserved
                 when the system is restarted. Ignored for all other commands,
                 which always affect the appropriate persistent routes.

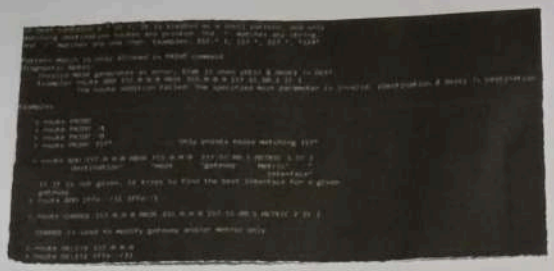
    -4           Force using IPv4.

    -6           Force using IPv6.

    command      One of these:
                   PRINT     Prints a route
                   ADD       Adds   a route
                   DELETE    Deletes a route
                   CHANGE    Modifies an existing route
    destination  Specifies the host.
    MASK         Specifies that the next parameter is the 'netmask' value.
    netmask      Specifies a subnet mask value for this route entry.
                 If not specified, it defaults to 255.255.255.255.
    gateway      Specifies gateway.
    interface    the interface number for the specified route.
    METRIC       specifies the metric, ie. cost for the destination.

All symbolic names used for destination are looked up in the network database
file NETWORKS. The symbolic names for gateway are looked up in the host name
database file HOSTS.

If the command is PRINT or DELETE. Destination or gateway can be a wildcard,
(wildcard is specified as a star '*'), or the gateway argument may be omitted.
```



Result

Thus the study of various network commands used in Linux and Windows is completed successfully.

Ex 8: Cisco Packet Tracer & Network Device Application

Date: 28/7/26

AIM:

To study the Packet tracer tool Installation and User Interface Overview

INTRODUCTION

1. It allows you to model complex systems without the need for dedicated equipment.

2 It is available for both Linux and Windows Desktop environments.

INSTALLING PACKET TRACER:

To download Packet Tracer, go to https://www.netacad.com and log in with Cisco Academy credentials.

1) Windows

Installation is pretty simple and the setup comes in a single file named Packettracer_Setup.

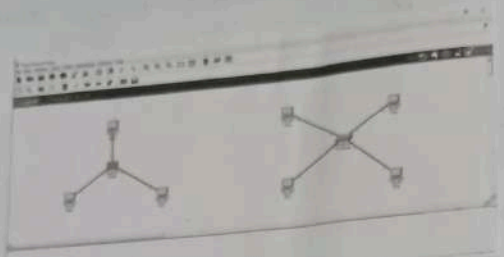
2) Linux

Linux users with an Ubuntu/Debian distribution should download the file for Ubuntu.

USER INTERFACE OVERVIEW

The components of the Packet Tracer Interface are follows:

1) Menu Bar: It is a common menu found in all software applications. It is used to open, save, print and so on.



PC0

Physical | Config | Desktop | Programming | Attributes

IP Configuration

Interface: FastEthernet0

IP Configuration

DHCP | Static

IPv4 Address: 10.1.1.1

Subnet Mask: 255.0.0.0

Default Gateway: 0.0.0.0

DNS Server: 0.0.0.0

PC1

Physical | Config | Desktop | Programming | Attribu

IP Configuration

Interface: FastEthernet0

IP Configuration

DHCP | Static

IPv4 Address: 10.1.1.2

Subnet Mask: 255.0.0.0

Default Gateway: 0.0.0.0

DNS Server: 0.0.0.0

3) Main ToolBar:
to menu options
open, save, zoom, und

3) logical work
you to toggle
work areas

1) Workspa
topologies
are displ

5) Comm
controls

6) Ro
used to

7) N
contain
availab

Creat
thei

ANALYSE T

1. F

2.

2) Main Toolbar: This bar provides shortcut icons to menu options that are commonly accessed such as open, save, zoom, undo and redo.

3) Logical workspace tabs: These tabs allow you to toggle between the logical and Physical work areas

4) Workspace: This is the area where topologies are created and simulations are displayed.

5) Common tools bar: This toolbar provides controls for manipulating topologies.

6) Real-time / Simulation Tabs: These tabs are used to toggle between the real and simulation

7) Network Component Box: This component contains all of the network and end devices available with Packet Tracer.

8) User-created packet Box: Users can create highly customized packets to test their topology from this area.

ANALYSE THE BEHAVIOUR

1. From the network component box, click
   a) 4 Generic PCs & one HUB
   b) 4 Generic PCs & One Switch

2. Click on Connections
   a) Click on Copper Straight Through cable

c) Similarly connect 4 PCs to the switch using copper straight through cable

5) Click on PCs connected to hub go to the Desktop tab, click on IP configuration & enter an IP address & subnet mask.

4) Observe the flow of PDU from source PC to destination PC by selecting Realtime mode

5) Repeat step 3 to step 5 for the PCs connected to switch

4) Observe how HUB and switch are forwarding the PDU.

Result

To study the packet tracer tool installation and User Interface is completed successfully

Ex: 2

Date: 25/7/26

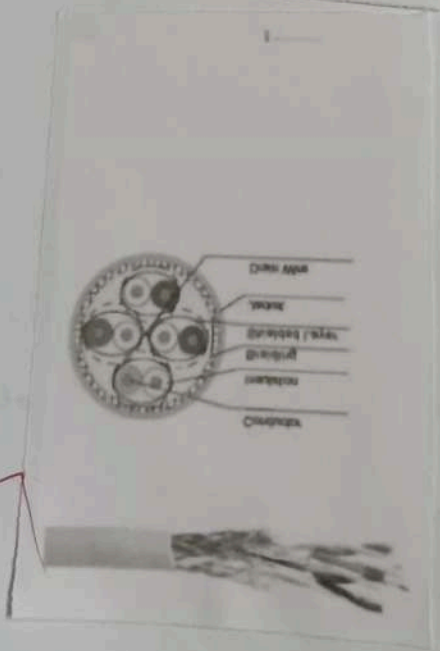
AIM: Study of

a) Understand di

1. Un
2. S
3. C
4.

Cable Type	Category	Max T
UTP	Category 3	10
	Category 5	Up t
	Category 5e	1
STP	Category 6, 6a	
SSTP	Category 7	
Coaxial cable	RG-6 RG-59 RG-11	
Fibre Optic Cable	Single mod Multi mod	





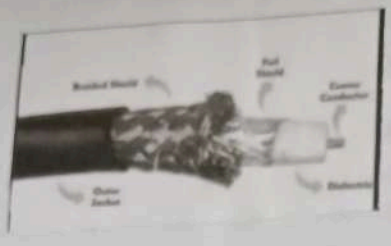
Ex. 2 Study of Network Cables

Date. 25/7/26

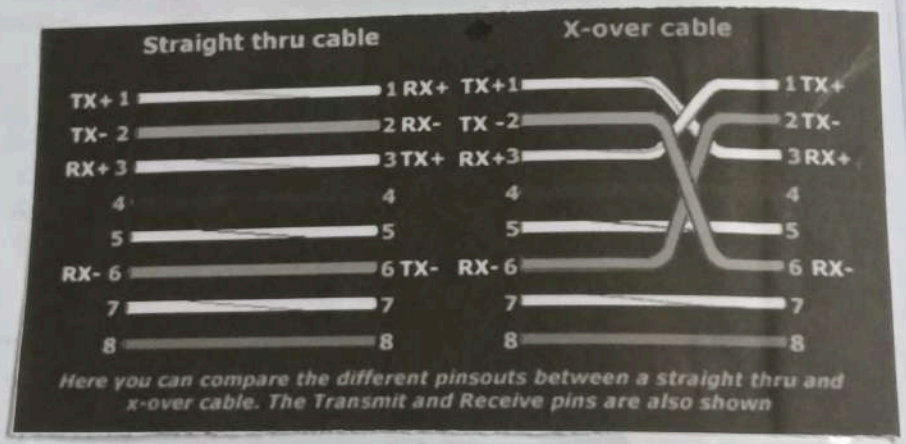
AIM: Study of different types of network cables.

a) Understand different types of network cable
1. Unshielded Twisted Pair (UTP) Cable
2. Shielded Twisted Pair (STP) cable
3. Coaxial Cable
4. Fibre Optic Cable

Cable Type	Category	Maximum Data Transmission	Advantages / Disadvantages	Application /use
UTP	Category 3	10 bps	Advantages * Cheaper in cost * Easy to install as they have a smaller overall diameter Disadvantages * More prone to (EMI) Electromagnetic interference & noise	10 Base - T Ethernet
	Category 5	Up to 100 Mbps		Fast Ethernet. Gigabit Ethernet
	Category 5e	1 Gbps		Fast Ethernet. Gigabit Ethernet
STP	Category 6, 6a	10 Gbps	Advantages * Shielded * Faster than UTP * Less susceptible to noise & interference Disadvantages * Expensive * Greater installation effort	Gigabit Ethernet, 10G Ethernet (55 m) Widely used in data centres
SSTP	Category 7			Gigabit Ethernet, 10G Ethernet (100m)
Coaxial Cable	RG-6 RG-59 RG-11	10-100 Mbps	Advantages * High bandwidth * Immune to interference * Low loss bandwidth Disadvantages * Limited distance * Cost * Size is bulky	- Speed of signal is 500 m - Television network - High speed internet connections
Fibre Optic Cable	Single mode Multi mode	100 Gbps	Advantages * High speed & security * High bandwidth * Long distance Disadvantages * Expensive * Requires skilled installers	Maximum distance of fibre optics cable is around 100 meters







Here you can compare the different pinsouts between a straight thru and x-over cable. The Transmit and Receive pins are also shown



b) Make Your Own

Tools & parts

* Ethernet ca
support, but CA
shorter distanc

* A crimpin
networking tool
in the plug and

* Two RJ45

Step 1: To st
by three

Step 2: Next
from both
round are

Step 3: Afte
wires th
Referencing
top to
arrangem

Step 4: On
together in
stick out
create a
placing
messing
with
have

Step 5: M
of p
shiel

b) Make Your Own Ethernet Cross-Over Cable

Tools & parts needed:

* Ethernet cabling. CAT5e is certified for gigabit support. But CAT5 cabling works as well, just over shorter distances.

* A crimping tool. This is an all in one networking tool shaped to push down the pins in the plug and strip and cut shielding off the cables.

* Two RJ45 plugs * Optional two plug shields.

Step 1: To start construction of the device begin by threading shields onto the cable

Step 2: Next strip approx 1.5 cm of cable shielding from both ends. The crimping tool has a round area to complete this task

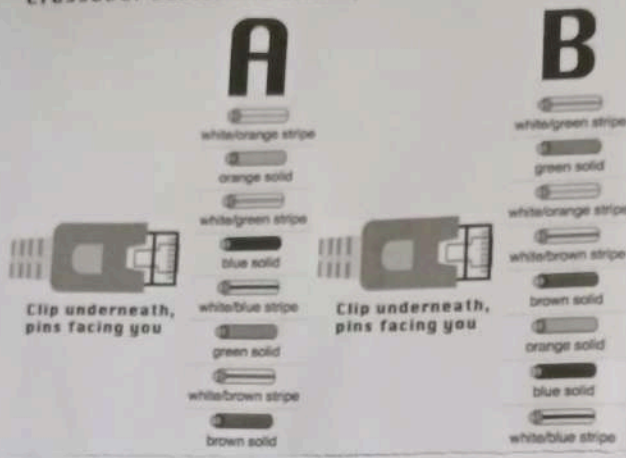
Step 3: After you will need to untangle the wires there should be four 'twisted pairs'. Referencing back to sheet, arrange them from top to bottom. One end should be in arrangement A and the other in B.

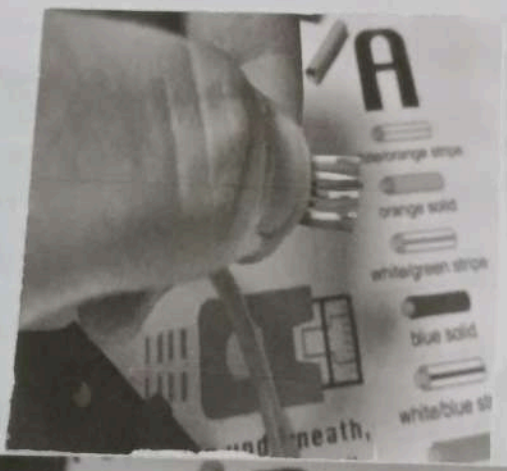
Step 4: Once the order is correct, bunch them together in a line, and if there are any that stick out farther than others strip them back to create an even level. The difficult aspect is placing these into the RJ45 plug without messing up the order. To do so, hold the plug with the clip side facing away from you and have the gold pins facing toward you.

Step 5: Next, push the cable. The notch at the end of plug needs to be just over the cable shielding

1 TX+
2 TX-
3 RX+
4
5
6 RX-
7
8
and

Straight through network cable: both sides should be A
Crossover cable: One side A, one side B







Step 6: After the wires are securely sitting inside the plug insert it into the crimping tool

Step 7: Repeat for the other end using diagram B/L.

Result

study of different types of network cables is completed successfully.

Ex: 4 Setup and configure LAN using switch and Ethernet Cables

Date: 30/7/24

AIM: Setup LAN using switch and Ethernet

What is a LAN?

A Local Area Network (LAN) is a network that connects devices within a small area such as home, school or office. It allows users to share resources like files, printers and internet access. A network switch connects the devices and enables fast and secure communication between them.

How to setup a LAN?

Step 1: Plan the network topology and arrange the required devices.

Step 2: Connect 4 computers to a network switch using Ethernet cables.

Step 3: Assign static IP addresses to all computers

- PC1 : 10.1.1.1
- PC2 : 10.1.1.2
- PC3 : 10.1.1.3
- PC4 : 10.1.1.4
- Subnet Mask : 255.0.0.0

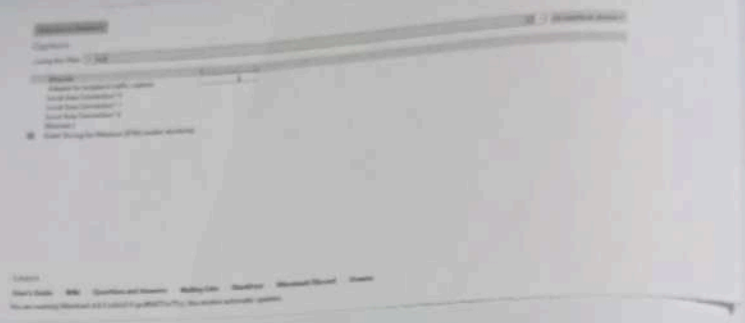
Step 4: Configure the switch and assign it the IP address 10.1.1.5 with subnet mask 255.0.0.0

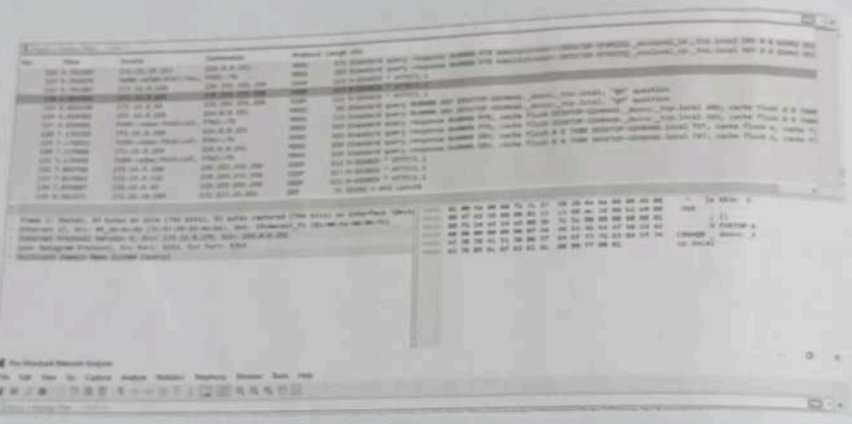
Step 5: Verify network connectivity using the ping command.

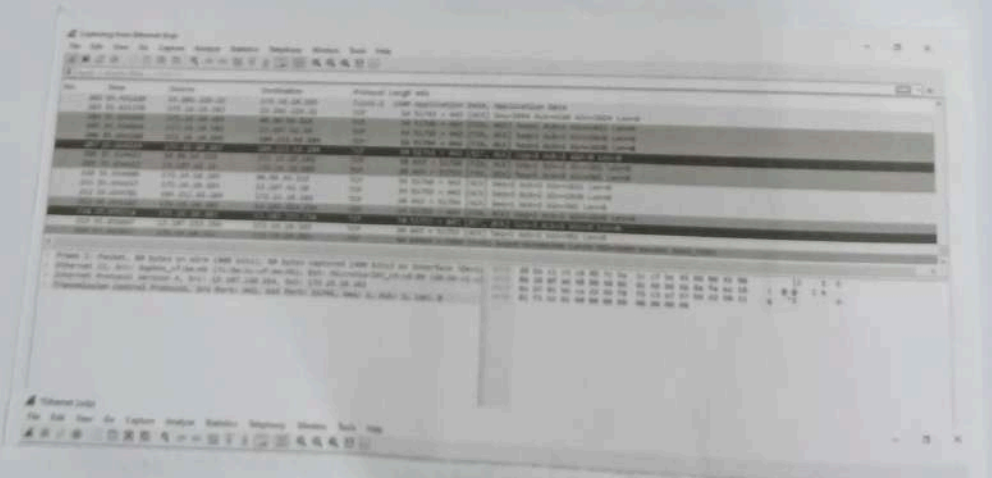
Step 6: Share a folder on one computer and access it from other computers on the LAN to confirm successful file sharing.

Result

To setup LAN using switch and Ethernet is completed successfully.







Ex No:5 EXPER

Date: 1/8/25

Aim

To capture, ...
packets from th...
wireshark pac...

OBJECTIVES:

1. To understand ...
Packet sniffers

2. To capture ...
mode on a ...

3. To apply ...
protocol inf...

4. To analyze ...

5. To inspect ...
stream co...

SOFTWARE

* System :

* Software ...

* Network ...

PROCEDURE

G...

1. Laun...

2. Sele...
inte...

3. Go ...
a...

Ex No:5 EXPERIMENTS ON PACKET CAPTURE TOOL

Date: 1/8/26

AIM

To capture, filter, view and analyze network packets from the on Ethernet interface using the wireshark packet sniffer tool.

OBJECTIVES:

1. To understand the structure and functioning of Packet sniffers like wireshark and tcpdump.
2. To capture active network traffic using promiscious mode on a local interface.
3. To apply display filters for specific network protocol including TCP, UDP, ARP, DNS, HTTP, ICMP and DHCP.
4. To analyze packet, structure
5. To inspect TCP flow graphs and full packet stream conversations.

SOFTWARE REQUIRED:

* System: Computer with windows/ Linux / macOS
* Software: Wireshark Network Analyzer
* Network Interface: Active Ethernet LAN or wifi interface.

PROCEDURE:

General Packet Capture Procedure

1. Launch wireshark on your system
2. Select the active Local Area Connection / Wi-fi interface
3. Go to capture → options and select "stop capture automatically after 100 packets".

4. Click start capture
5. After capturing 100 packets, save the captured file for analysis.

PROTOCOL FILTERING PROCEDURES:

1. TCP/UDP packets:

* Enter tcp or UDP in the wireshark filter bar
* To view the communication flow, click statistics

2. ARP Packets

* Enter arp in the filter search bar to display Address resolution protocol packets.
* Inspect the hardware and protocol addresses in the packet Details pane.

3. DNS packets

* Enter DNS in the filter search bar to capture domain name system queries and responses.
* View the conversation flow via statistics → flow graph

4. HTTP Packets

* Enter http in the display filter book
* Inspect web request and response header details

5. IP/ICMP Packets

Enter ICMP or IP in the filter bar to observe ping/echo traffic

6. DHCP packets

Enter DHCP in the filter bar to view dynamic IP address assignments

Result:

1. The Wireshark packet capturing tool was successfully configured and operated

2. Network packets were captured across the ethernet interface and specific protocols were isolated and analyzed using display filters

Ex No : 6 ERROR DETECTION AND CORRECTION USING HAMMING CODE

Date : 4/8/26

AIM :

To write a program to implement error detection using Hamming Code.

ALGORITHM

1. Start the program
2. Read number of 4-bit data blocks
3. For each 4-bit data block D = d_1 d_2 d_3 d_4
   a) Arrange 7 bits with parity bits at position 1, 2, 4 and data bits at 3, 5, 6, 7
   b) Compute parity bits
      $P_1 = d_1$ XOR d_2 XOR d_4 (covers 1, 3, 5, 7 bits)
      $P_2 = d_1$ XOR d_3 XOR d_4 (covers 2, 3, 6, 7 bits)
      $P_4 = d_2$ XOR d_3 XOR d_4 (covers 4, 5, 6, 7 bits)
   c) Form 7 bit Hamming Code : P_1 P_2 d_1 P_4 d_2 d_3 d_4
4. Simulate transmission by introducing a single bit error at chosen bit position
5. At receiver, recompute check bits
   $C_1 = P_1$ XOR d_1 XOR d_2 XOR d_4
   $C_2 = P_2$ XOR d_1 XOR d_3 XOR d_4
   $C_4 = P_4$ XOR d_2 XOR d_3 XOR d_4
6. Compute error position = C_4 C_2 C_1
7. If Error position = 0, no error occurd
   Else the bit is inverted to correct error
8. Extract the original 4 data bits from the corrected 7-bit code word
9. Repeat Steps 3 to 8 for all n blocks in the data stream
10. Stop the program

PROGRAM

```
def encode(d):
    h = [0] * 7
    h[2] = d[0]
    h[4] = d[1]
    h[5] = d[2]
    h[6] = d[3]

    h[0] = h[2] ^ h[4] ^ h[6]
    h[1] = h[2] ^ h[5] ^ h[6]
    h[3] = h[4] ^ h[5] ^ h[6]
    return h

def detect_and_correct(h):
    c1 = h[0] ^ h[2] ^ h[4] ^ h[6]
    c2 = h[1] ^ h[2] ^ h[5] ^ h[6]
    c3 = h[3] ^ h[4] ^ h[5] ^ h[6]
    error_pos = c1*1 + c2*2 + c4*4
    if error_pos != 0:
        print(f"Error detected at bit position: {error_pos}
                                    → Correcting...")
        h[error_pos - 1] ^= 1
    else:
        print("No error detected.")
    return error_pos

def to_str(bits):
    return "".join(str(b) for b in bits)

def main():
    print("::: HAMMING CODE - ERROR DETECTION AND
                                    CORRECTION :::\n")
    n = int(input("Enter number of 4-bit data blocks in
                                    the stream: "))
    for b in range(1, n+1):
        print(f"\n--- Block {b} ---")
        d = list(map(int, input("Enter 4 data bits
            (space separated, e.g. 1 0 1 1): ").split()))
        h = encode(d)
        print(f"Original Data Bits: {to_str(d)}")
        print(f"Encoded Hamming Code (7 bits): {to_str(h)}")
```

OUTPUT

... HAMMING CODE - ERROR DETECTION AND CORRECTION

Enter number of 4-bit data blok in the stream : 2

--- Block 1 ---

Enter 4 data bits (space separated, e.g. 1 0 1 1): 1 0 1 1

Original data bits : 1011

Encoded Hamming Code (7 bits): 0110011

Enter bit position to introduce error (1-7, 0 = no error): 3

Transmitted Code (with error included at pos 3): 0100011

Received Code : 0100011

Checking for errors...

Error detected at bit position : 3 → Correcting...

Corrected Code : ~~1100110~~ 0110011

Recovered Data bits : 1011

--- Block 2 ---

Enter 4 data bits (space separated, e.g. 1 0 1 1): 0 1 1 0

original data bits : 0110

Encoded Hamming Code (7 bits) : 1100110

Enter bit position to introduce error (1-7, 0 - no error): 6

Transmitted Code (with error included at pos 6):
1100100

Received Code : 1100100

Checking for errors...

Error detected at bit position : 6 →
Correcting...

Corrected Code : 1100110

Recovered Data bits : 0110

RESULT

The

correct

success

AND CORRECTION...

stream : 2

: 1011

ror): 3

0100011

recting...

g101,): 0110

0 = no error): 6

at pos 6):

0100

; 6 →

rrecting...

```
pos = int(input("Enter bit position to introduce error (1-7, 0 =
                                                    no error): "))

if 1 <= pos <= 7:
    h[pos-1] ^= 1
    print(f"Transmitted Code (with error induced at pos {pos}):"
          f" {to_str(h)}")
else:
    print(f"Transmitted Code (no error induced): {to_str
                                                  (h)}")

    print(f"Received Code : {to_str(h)}")
    print("Checking for errors...")
    detect_and_correct(h)
    print(f"Corrected Code: {to_str(h)}")

    rd = [h[2], h[4], h[5], h[6]]
    print(f"Recovered Data Bits : {to_str(rd)}")

if __name__ == "__main__":
    main()
```

RESULT

The program to implement error detection & correction using Hamming Code is completed successfully in python.

Ex No: 7 FLOW CONTROL AT DATA LINK LAYER USING SLIDING WINDOW PROTOCOL

Date: 8/8/24

AIM:

To write a program to implement flow control at the Data Link layer using the Sliding Window Protocol and to simulate the flow of frames from one node to another.

ALGORITHM

1. Start the program
2. Read the total number of frames to be sent (N), the sender's window (W) & a frame number to simulate
3. Initialize base = 0 & nextSeq = 0
4. Repeat while base < N:
   a) Determine current sender window as range
   b) Send all frames with sequence numbers from nextSeq up to incrementing nextSeq after each frame.
   c) If the lost frame lies within frames sent
      (1) The receiver discards the lost frame & any subsequent out-of-order frames
      (2) The sender times out & retransmits all frames starting from lost frame
   d) Else
      (1) The receiver accepts each frame in order & sends a acknowledgement (ACK)
      (2) On receiving ACK for the lost frame sent the sender slides its window forward base = nextSeq.
5. Repeat step 4 until all N frames have been transmitted & acknowledged
6. Stop the program

OUTPUT:

::: SLIDING WINDOW PROTOCOL - FLOW CONTROL SIMULATION (Go-Back-N) :::

Enter total number of frames to send : 8

Enter sender's window size : 4

Enter frame number to simulate as lost

(0 to 7, enter -1 for no loss) : 5

Sender Window : [Frame 0 - Frame 3]

Node A -> Sending Frame 0

Node A -> Sending Frame 1

Node A -> Sending Frame 2

Node A -> Sending Frame 3

Node B -> Received Frame 0, sending ACK 0

Node B -> Received Frame 1, sending ACK 1

Node B -> Received Frame 2, sending ACK 2

Node B -> Received Frame 3, sending ACK 3

Node A -> ACK 3 received. Sliding Window forward

Sender Window : [Frame 4 - Frame 7]

Node A -> Sending Frame 4

Node A -> Sending Frame 5

(Frame 5 LOST in transit!)

PROGRAM

```
def simulate(
    base = 0
    next_seq = 0
    while base
        windo
        print
        print
    wh
```

PROGRAM

```
def simulate(total_frames, window_size, lost_frame):
    base = 0
    next_seq = 0
    while base < total_frames:
        window_end = min(base + window_size - 1, total_
                                                frames - 1)
        print("\n ----------")
        print(f"Sender Window: [Frame {base} - Frame
                                        {window_end}]")
        while next_seq < base + window_size and next_seq
                                        < total_frames:
            print(f"Node A --> Sending Frame {next_seq}")
            if next_seq == lost_frame:
                print(f"(Frame {next_seq} LOST in transit!)")
            next_seq += 1

        if lost_frame is not None & base <=
                                    lost_frame < next_seq:
            print(f"Node B --> Frame {lost_frame}
                missing. Discarding out-of-order frames
                                        after it")
            print(f"Node A --> Timeout! Resending
                window starting from Frame {lost_
                                        frame}.")
            next_seq = lost_frame
            lost_frame = None
            continue
        for f in range(base, next_seq):
            print(f"Node B -> Received Frame {f},
                                    sending ACK {f}")
        print(f"Node A --> {next_seq - 1} received
            Sliding window forward")
        base = next_seq

    print("\n ----------")
    print(f"All {total_frames} frames
        successfully transmitted from Node A to B.")
```

Node A -> Sending Frame 6

Node A -> Sending Frame 7

Node B -> Frame 5 missing. Discarding out-of-order frames after it.

Node A -> Timeout! Resending window starting from Frame 5

Sender window : [Frame 4 - Frame 7]

Node A -> Sending Frame 5

Node A -> Sending Frame 6

Node A -> Sending Frame 7

Node B -> Received Frame 4, sending ACK 4

Node B -> Received Frame 5, sending ACK 5

Node B -> Received Frame 6, sending ACK 6

Node B -> Received Frame 7, sending ACK 7

Node A -> ACK 7 received, Sliding window forward

All 8 frames successfully transmitted from Node A to Node B

```
def main():
    print("== SLIDING WINDOW PROTOCOL - FLOW CONTROL
                SIMULATION (Go-Back-N) ==\n")
    total_frames = int(input("Enter total number of
                frames to send  :"))
    window_size = int(input("Enter sender's window
                size  :"))
    lost_frame = int(input("Enter frame number
                to simulate as lost\n (0 to %total_
                frames-1 %, enter -1 for no loss)  :"))
    if lost_frame == -1:
        lost_frame = None
    simulate(total_frames, window_size, lost_frame)

if __name__ == "__main__":
    main()
```

RESULT:

The program to implement flow control at the Data Link Layer using the sliding Window Protocol was successfully written in Python, executed and verified by simulating the flow of frames between two nodes (Node A as sender and Node B as receiver)

Ex No: 8 a LIVE HOST DISCOVERY USING NMAP SCANS

Date: 13/8/24

AIM:

To discover live hosts on a network using various nmap host discovery scans on the TryHackMe platform

PROCEDURE:

1. Connect the TryHackMe network using the OpenVPN configuration file (or) start the browser based AttackBox

2- Define target hosts using one of following

* Individual IPs / Hostnames : MACHINE-IP
* IP Range : 10.11.12.15-20
* Subnet (CIDR) : 10.10.12.0/24

3. ARP (SCAN)

↳ Use an ARP scan when the target is on the same local Ethernet subnet to discover MAC addresses without performing posts scans

4. ICMP scans:

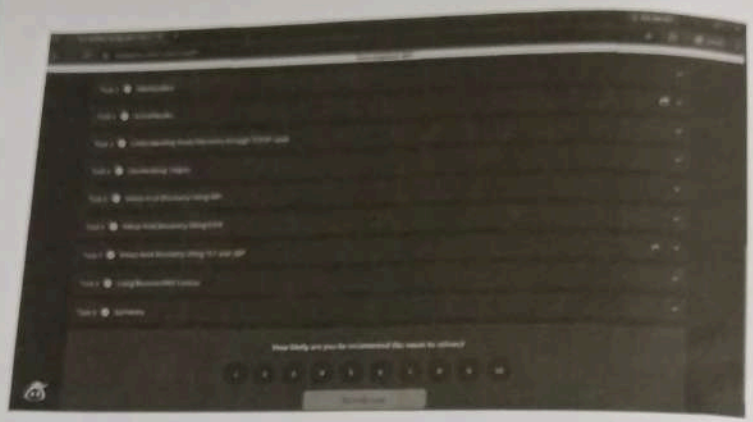
↳ ICMP Echo ping
↳ ICMP Timestamp ping
↳ ICMP Address Mask ping

5. TCP ping scans:

↳ Target port 23
↳ Target fault port 80
↳ TCP Ack ping

6. UDP ping scan

↳ send UDP packets to closed ports to checks for ICMP port unreachable replies



PC0

Physical Config Desktop Programming Attributes

Command Prompt

```
Cisco Packet Tracer PC Command Line 1.0
C:\>tracert 30.0.0.20

Tracing route to 30.0.0.20 over a maximum of 30 hops:

  1    0 ms      0 ms      0 ms      10.0.0.1
  2    *         0 ms      0 ms      20.0.0.2
  3    *         0 ms      0 ms      30.0.0.20

Trace complete.

C:\>tracert 30.0.0.100

Tracing route to 30.0.0.100 over a maximum of 30 hops:

  1    0 ms      0 ms      0 ms      10.0.0.1
  2    *         0 ms      0 ms      40.0.0.2
  3    *         0 ms      0 ms      20.0.0.2
  4    *         0 ms      0 ms      30.0.0.100

Trace complete.

C:\>
```

RESULT

success

UDP

each

7- Reverse - DNS Lookup

probem reverse DNS queries on all potential - on all hosts within a subnet to gather hostname insights

RESULT:

The live hosts on the target subnet were successfully identified using Nmap, ARP, ICMP, TCP and UDP host discovery scanning techniques - The behaviour of each protocol was verified and recorded

Ex No: 8b
Date 20/8/26

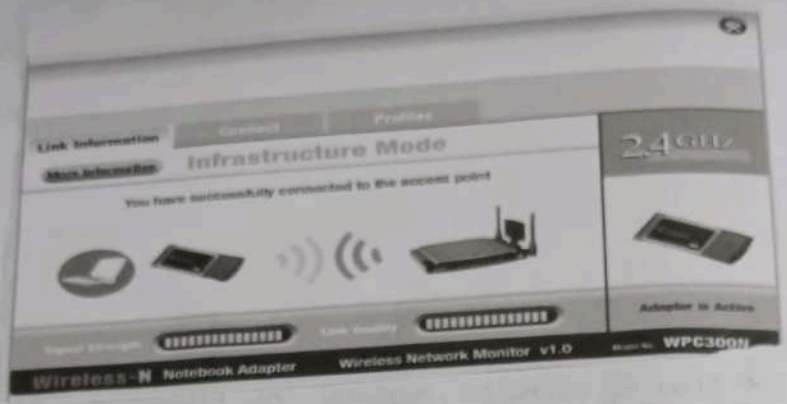
Wireless LAN Configuration

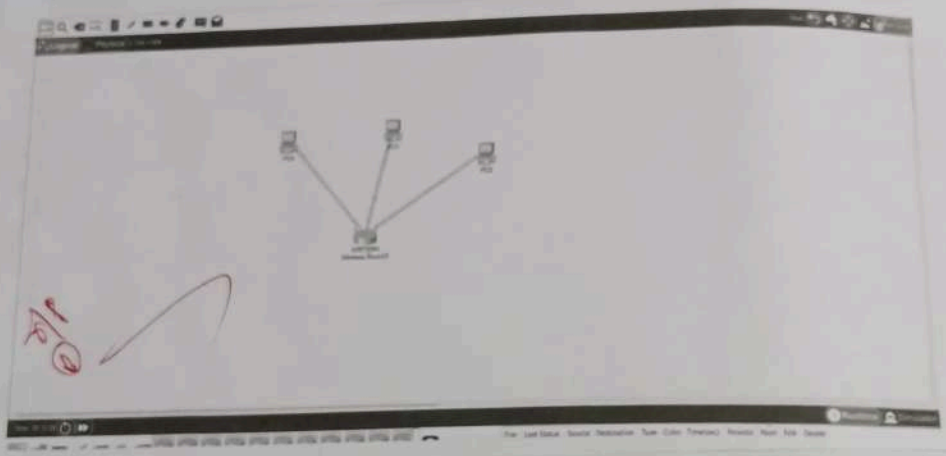
AIM:

To configure a wireless LAN (WLAN) connecting 3PCs to a linksys wireless Router with static IPS and WEP security in Cisco Packet Tracer

PROCEDURE

1. Topology Setup
   * Place 1 linksys wireless Router & 3 PCs
   * Turn off each PC, replace the Ethernet module with a wireless Module & turn the PC back on
2. Router Setup
   * Open Router GUI → Administration tab → setup username & password to admin → click save setting
   * Go to wireless tab → set SSID to Mother Network
3. static IP Assignment
   * Router: 192.168.0.1
   * PC0: IP 192.168.0.2 / Subnet 255.255.255.0 / Gateway 192.168.0.1
   * PC1: IP 192.168.0.3 / Subnet 255.255.255.0 / Gateway 192.168.0.1
   * PC2: IP 192.168.0.4 / Subnet 255.255.255.0 / Gateway 192.168.0.1
4. Wireless Connection
   * On each PC, open Desktop → PC wireless → connect tab → click refresh
   * Select Mother Network, click network, enter WEP key 0123456789 and click connect





Ex No: 9
Date: 29/8/24

AIM:

To implement
switches and PCs

PROCEDURE:

1. Topology Setup
   - Place 2 R
   - CPCI, PCs fo
   - Connect
   using Ethern
2. Subnet Calc
   - Subnet
   - Subnet
3. Router Conf
   - Open

4. PC Con
   - Op

RESULT:

The wireless LAN topology was designed secured using WLP key and all 3 PCs successfully connected to Mothernetwork

Ex No: 9
Date: 29/8/26

Implementation of Subnetting

AIM:

To implement classless IP subnetting using routers, switches and PCs in Cisco Packet Tracer & verify connectivity.

PROCEDURE:

1. Topology Setup:
   - Place 2 Routers (R_1, R_2), 2 Switches (S_1, S_2) and PCs (PC_1, PC_5 for each subnet)
   - Connect routers to switches and switches to PCs using Ethernet cables.
2. Subnet Calculation (/27 Mask).
   - Subnet mask: 255.255.255.224 (/27)
   - Subnet 1: 192.168.1.0/27 (Gateway: 192.168.1.1)
3. Router Configuration (R_1 Example):
   - Open router CLI & type

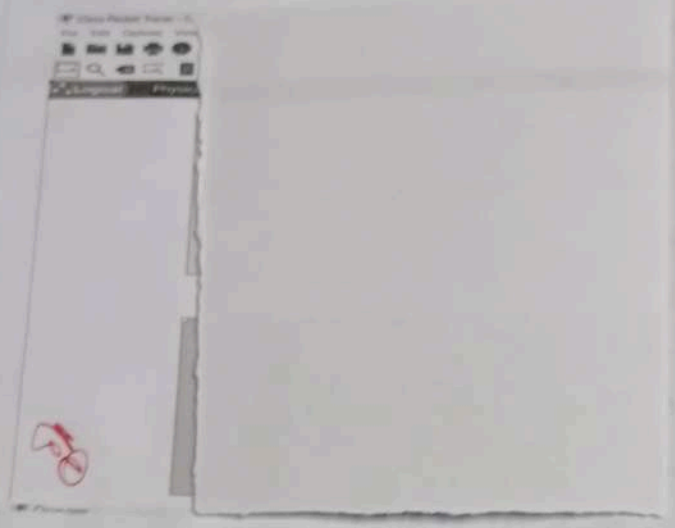
```
enable
configure terminal
interface GigabitEthernet 0/0
ip address 192.168.1.1 255.255.255.224
no shutdown
exit
```

4. PC Configuration
   - Open each PC → Desktop → IP Configuration → Static
     - *PC1: IP 192.168.1.11 | Subnet 255.255.255.224/
       Gateway 192.168.1.1
     - *PC2: IP 192.168.1.12 | Subnet 255.255.255.224/
       Gateway 192.168.1.1

5. Testing

Open Command prompt on PC1 and test connection

ping 192.168.1.1



RESULT:

Classless IP Subnetting (/27 mask) was implemented successfully in Cisco Packet Tracer, and network communication was verified using the ping command.

Ex No: 10(a) Internet working with Router

Date: 10/9/26

AIM:

To connect two PCs in different networks using a router in Cisco packet Tracer & verify connection using PDU packet

PROCEDURE

Step 1: Set up Devices

* Add 1 Router & 2 PCs (PC0, PC1) to the work space
* Connect them using Copper_through straight Cables.

Step 2: Configure PCs

* PC0 ↳ IP address: 192.168.10.2
  Subnet Mask: 255.255.255.0
  Default Gateway: 192.168.10.1
* PC1 ↳ IP address: 192.168.20.2
  Subnet mask: 255.255.255.0
  Default Gateway: 192.168.20.1

Step 3: Configure the router

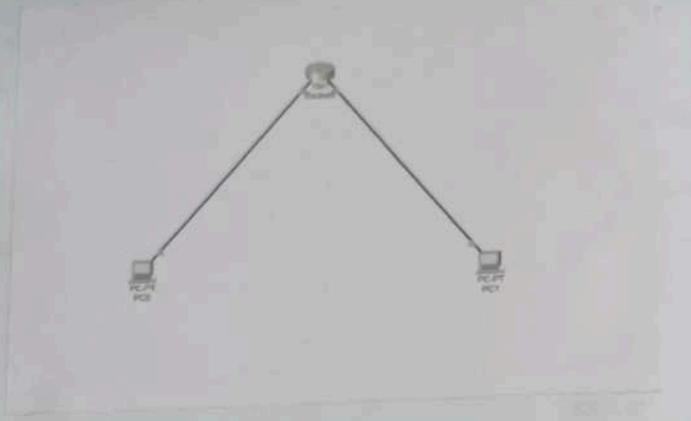
```
enable
config t
interface Fast Ethernet 0/0
ip address 192.168.10.1 255.255.255.0
no shut down
exit
interface Fast Ethernet 0/1
ip address 192.168.20.1 255.255.255.0
no shutdown
exit
```

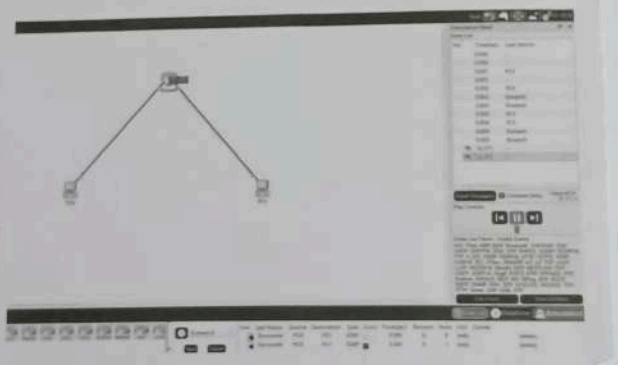
Step 4: Test Connection

* Click pc0 → command promt → type ping

192.168.20.2

8. Click add the simple PDU → Click PC0 → Click PC1 to see the successful packet Transfer.





RESULT:

The router network was successfully created and tested by sending a PDU form PC0 to PC1

Ex.No: 10(b) Internet
Date: 13/9/26 DHCP ser

AIM:

To design, configur
a wireless router, lap
and a DHCP/DNS serv

PROCEDURE

Step 1: Connect the to

* Place the dev
them using one co
* PC → wireless
* Cable modem

Step 2: Configure th

* 1 Click the wirel
• Set netw
* 2 Go to the se
• Ensure D
• Set stat

Step 3. Configure
* Click laptop

Step 4: Configure

Step 5: Configure
* Service
* Static

Step 6: Test con

* Run ipcon
* Run ping

PCO → click PC

Ex No: 10(b) Internetworking with wireless router, DHCP server and Internet cloud

Date: 13/9/24

AIM:

To design, configure and test internetwork using a wireless router, laptop, PC, cable, modem, Internet cloud and a DHCP/DNS server in Cisco Packet tracer.

PROCEDURE

Step 1: Connect the topology

* Place the device in the workplace and connect them using the correct cables
* PC → wireless router * Wireless router → cable modem
* Cable modem → Internet cloud * Internet cloud → Cisco.com server

Step 2: Configure the wireless router

*1 Click the wireless router → GUI tab → Wireless tab
• Set network name (SSID): Home network

*2 Go to the setup tab
• Ensure DHCP server is enabled
• Set static DNS1: 208:67.220.220

Step 3: Configure the laptop & PC

* Click laptop - physical tab → Turn off power switch

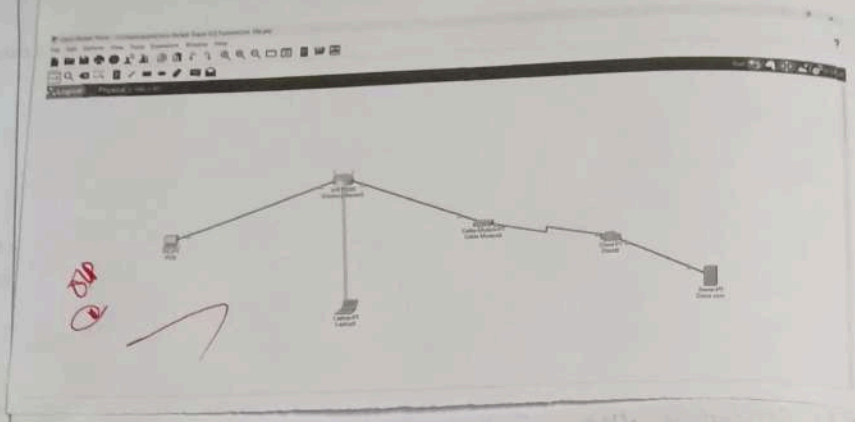
Step 4: Configure the Internet cloud

Step 5: Configure the Cisco.com server

* Service configuration
* Static IP configuration

Step 6: Test connectivity

* Run ipconfig /renew to fetch dynamic
* Run ping Cisco.com to configure domain connectivity



RESULT

The network with topology, router, cable modem, Internet cloud and DHCP/DNS server was successfully built and configured. Connectivity was verified by resolving and pinging cisco.com from the PC

Ex No: 11(a)
Date: 17/9/26

AIM:

To simulate
primary and ba

PROCEDURE:

1. Design a n
by connecting
host machines
serial cables.

2. Assign valid
masks and
all client co

3. Determine
each router
in the routin

4 - Assign a
to the prim
distance v
it as a f

5. Inspect th
primary pa
route sele

ping utilit

Ex No: 11(a) Static Routing Protocol Configuration
Date: 17/9/24

AIM:

To simulate static routing configuration with primary and backup routes using Cisco Packet Tracer

PROCEDURE:

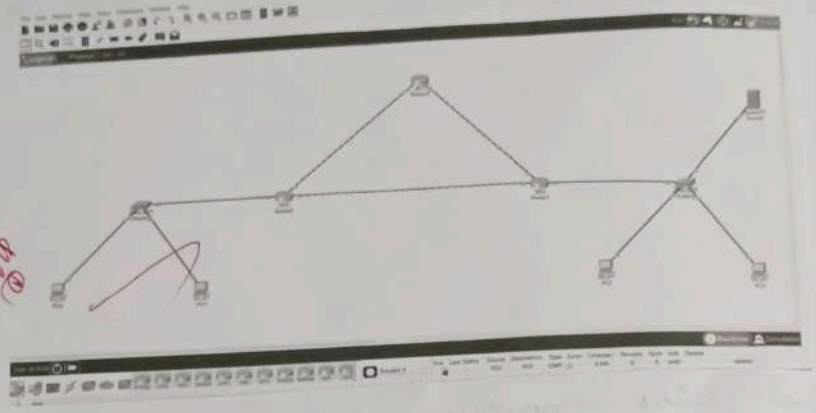
1. Design a network topology in the simulator by connecting multiple routers, switches and client host machines using appropriate Ethernet and serial cables.

2. Assign valid IP addresses, appropriate subnet masks and default gateway configurations to all client computers and active router interfaces.

3. Determine the unadvertised remote networks for each router and manually define static routes in the routing tables.

4. Assign a lower administrative distance value to the primary link and a higher administrative distance value to the alternate link to configure it as a floating backup route

5. Inspect the static routing table to verify that only primary paths are installed and test connectivity and route selection across subnets using trace and ping utilities.



RESULT:

Static routing protocol is successfully simulated with functional primary and automated backup paths.

Ex No: 11(b) Simulat

Date: 24/9/25

AIM:

To configure RIP

verify automatic pa

PROCEDURE:

Step 1: 1. Add 3 Routers

2. Connect PC0 → R

straight through

2. Connect Route

- Route
- Route
- Route

Step 2: Assign IP

Step 3: Configure R

- Open CLI or

- Router 0:

text
enable
config t
interface fa
no shutdown
interface se
clock rate
no shutdown
ip address
no shutdo

- Router 2

enable
config
interface
ip address
no shutdo

Step 4: Conf

Step 5: Test

Ex No: 11(b) Simulate RIP using CISCO Packet Tracer

Date: 24/9/25

AIM:

To configure RIP in Cisco Packet Tracer and verify automatic path switching during link failure

PROCEDURE:

Step 1:

1. Add 3 Routers and 2 PCs
2. Connect PC0 → Router0, PC1 → Router 2 using Copper straight through cables
3. Connect Routers using Serial DCE cable.
   - Router 0 → Router1
   - Router1 → Router2
   - Router0 → Router2

Step 2: Assign IP Addresses on PCs

Step 3: Configure Router Interfaces

. Open CLI on each router and run

- Router 0:

```
text
enable
config t
interface fa0/0
no shutdown
interface s0/0/0
clock rate 64000.
no shutdown
ip address 192.168.1.257
no shutdown
```

- Router 1

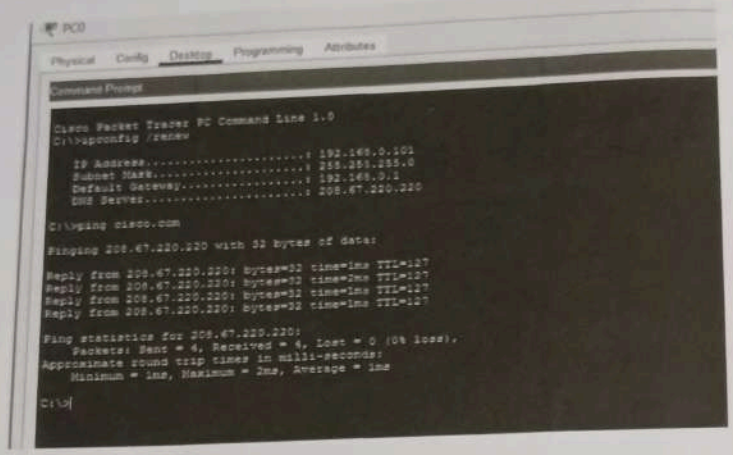
```
text
enable
config t
interface s0/0/0
no shutdown
ip address 192.168.1.246
clock rate 64000
no shutdown
```

- Router 2

```
enable
config
interface fa0/0
ip address 20.0.0.1
no shutdown
```

Step 4: Configure RIP Protocol

Step 5: Test Connectivity



RESULT

RIP routing protocol was configured and verified. Dynamic rerouting during link failure was confirmed using tracert.

Ex No: 12 a ECHO c
Date: 26/3/26

AIM:

To write and exec
implement a TCP Echo

ALGORITHM

Server Side

1. Create a socket
2. Bind the socket to
3. Listen for incomin
4. Accept the connec
5. Read / Receive the
6. Send / Echo the
7. close the client

Client Side

1. Create a socket
2. Connect to the
port number
3. Take input from
4 Send the mes
5. Receive the ec
6. Display the ec
7. Close the cli

CODE (JAVA)

Echo Server.ja

```
import jav
import jav
public cla
```

pu

Ex No: 10 a Echo client Server using TCP Sockets
Date: 28/9/26

AIM:

To write and execute a Java socket program to implement a TCP Echo client and Echo Server

ALGORITHM

Server Side

1. Create a socket
2. Bind the socket to an IP address & port number
3. Listen for incoming client connections
4. Accept the connection request from the client
5. Read / Receive the message sent by the client
6. Send / Echo the same message back to the client
7. Close the client & server sockets

Client Side

1. Create a socket
2. Connect to the server with server's IP address and port number
3. Take input from the user
4. Send the message to the server
5. Receive the echoed message back from the server
6. Display the echoed message on the screen
7. Close the client socket

CODE (JAVA)

Echo Server.java

```
import java.io.*;
import java.net.*;
public class EchoServer {
    public static void main(String[] args) {
        try {
            ServerSocket serverSocket = new ServerSocket(5000);
            Socket socket = ServerSocket.accept();
            System.out.println("client connected!")
```

```
BufferedReader in = new BufferedReader(new InputStreamReader(socket.
                                    getInputStream()));
PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
String inputLine = in.readLine();
System.out.println("Received from client: " + inputLine);
out.println(inputLine);
System.out.println("Echoed back to client.");
socket.close();
serverSocket.close();
}
catch (IOException e) {
    e.printStackTrace();
}
```

EchoClient.java

```
import java.io.*;
import java.net.*;
public class EchoClient {
    public static void main(String[] args) {
        try {
            Socket socket = new Socket("localhost", 5000);
            System.out.println("Connected to Echo Server!");
            BufferedReader userInput = new BufferedReader
                    (new InputStreamReader(System.in));
            PrintWriter out = new PrintWriter(socket.
                                    getOutputStream());
            System.out.print("Enter message to echo: ");
            String message = userInput.readLine();
            out.println(message);
            String echoResponse = in.readLine();
            System.out.println("Server Echo Response: " +
                                    echoResponse);
            socket.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

INPUT and OUTPU

Server Output

Echo Server

client Conne

Received fr

Echoed ba

client Output

Connected

Enter me

Server Ec

RESULT

The

successfully

server co

back to th

INPUT and OUTPUT

Server Output

Echo Server started. Waiting for client
Client Connected!
Received from client: Hello Computer Networks Lab
Echoed back to client

Client Output

Connected to Echo Server!
Enter message to echo: Hello Computer Networks Lab
Server Echo Response: Hello Computer Network Lab

RESULT

The TCP Echo Client Server program was successfully implemented, compiled and executed & the server correctly echoed the received message back to the client.

Ex No : 12(b)
Date : 1/10/24

Chat client - Server using TCP Sockets

AIM

To write and execute a Java Socket program to Implement a two way chat client and chat Server allowing real time text interaction between client and server.

ALGORITHM

Server Side

1. Create a socket & bind it to a designated port
2. Listen and accept ~~client~~ connection from client
3. Start a communication loop
4. Close socket connections when chat ends

Client Side

1. Create a socket and connect to the server IP and port
2. Start a communication loop
3. Close socket connections when chat ends

CODE

1. ChatServer.java

```
import java.io.*;
import java.net.*;
public class ChatServer {
    public static void main (String[] args) {
        try {
            ServerSocket serverSocket = new Server
                                        Socket (6000);
            System.out.println ("Chat Server running
                                on port 6000. Waiting
                                for client...");
            Socket socket = Server Saket . accept();
```

```
System.out.println("Client connected! Start chatting.");
BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
BufferedReader sysIn = new BufferedReader(new InputStreamReader(System.in));
String clientMsg = "", serverMsg = "";
while (!clientMsg.equalsIgnoreCase("stop")) {
    clientMsg = in.readLine();
    if (clientMsg == null || clientMsg.equalsIgnoreCase("stop")) {
        System.out.println("client disconnected the chat.");
        break;
    }
    System.out.println("client: " + clientMsg);
    System.out.print("server: ");
    serverMsg = sysIn.readLine();
    out.println(serverMsg);
}
e.printStackTrace();
}
}
```

2. Chat Client.java

```
import java.io.*;
import java.net.*;
public class ChatClient {
    public static void main(String[] args) {
        try {
            Socket socket = new Socket("127.0.0.1", 6000);
            System.out.println("Connected to chat server! Type 'stop' to exit.");
            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
```

```
String clientMsg = "", serverMsg = "";
while (!clientMsg.equalsIgnoreCase("stop")) {
    System.out.print("Client:");
    clientMsg = sysIn.readLine();
    out.println(clientMsg);
    if (clientMsg.equalsIgnoreCase("stop")) {
        break;
    }
    serverMsg = in.readLine();
    System.out.println("Server: " + serverMsg);
}
socket.close();
} catch (IOException e) {
    e.printStackTrace();
}
}
```

INPUT & OUTPUT

Server Console

Chat Server running on port 6000... Waiting for client
Client connected! Start chatting
Client: Hi Server, how are you?
Server: Hello Client! I am fine
Client disconnected the chat

Client Console

Connected to chat Server! Type 'Stop' to exit
Client: Hi Server, how are you?
Server: Hello client! I am fine
Client: Stop

RESULT

The TCP chat client Server program was successfully written, compiled, executed and allowing real time two way communication between the client & server.

Ex No: 13
Date: 8/10/26

AIM:

To implement
python sockets
source and a
the response

ALGORITHM

1. Import the
2. Get the
from user
3. Create a
4. Record the
to destination
5. Wait for
6. Calculate
response is
7. Display
8. Repeat
requests

CODE:

```
import socke
import time
host = input
ip = socket
s = Socket
s = settim
for i in
    ms
    s
    s
try:
```

Ex No : 13
Date : 8/10/26

Implementing Ping Program

AIM:

To implement a simple ping program using python sockets to check the connectivity between a source and a destination host by measuring the response time.

ALGORITHM

1. Import the socket and time modules
2. Get the destination IP address or hostname from user
3. Create a UDP socket and set a timeout
4. Record the starting time and sent a message to destination
5. Wait for a response from destination
6. Calculate the round-trip time (RTT) if response is received
7. Display the response time or timeout message
8. Repeat the process for a fixed number of requests and close the socket.

CODE:

```
import socket
import time
host = input ("Enter host:")
ip = socket.gethostbyname (host)
s = socket.socket (socket.AF_INET, socket.SOCK_DGRAM)
s = settimeout(2)
for i in range(4):
    msg = f"ping {i+1}
    stout = time.time()
    s = sendto(msg.encode(), (ip,7))

try:
    data, addr = s.recvfrom(1024)
```

OUTPUT

INPUT

Enter host : 127.0.0.1

OUTPUT

Reply from 127.0.0.1 : time = 0.15 ms

Reply from 127.0.0.1 : time = 0.12 ms

Reply from 127.0.0.1 : time = 0.14 ms

Reply from 127.0.0.1 : time = 0.11 ms

If no echo service is running

Enter host : 127.0.0.1

Requested time out

Requested time out

Requested time out

Requested time out

rtt = (time.time

print (f' Reply

except socket. time

print ("Requ

s. close()

RESULT

A si

using pyt

receive res

time to

```
rtt = (time.time() - start) * 1000
print(f"Reply from {ip}: time = {rtt:.2f} ms")
except socket.timeout:
    print("Request timed out")
s.close()
```

RESULT

A simple ping program was implemented using python UDP sockets to send messages, receive responses and calculate the round-trip time to test network connectivity.

Ex No: 14 Packet sniffing using Raw Sockets

Date: 10/10/26

AIM: To implement a packet sniffing program using RAW sockets in python to capture and analyze network packets such as TCP, ID, UDP and ICMP packets

ALGORITHM

1. Import the socket and struct modules
2. Create a RAW socket using socket. AF_INET and socket. SOCK_RAW
3. Bind the socket to local network interface if required
4. Capture incoming IP packets using the recv from() method
5. Extract the IP Header from captured packet
6. Identify the source IP, destination IP and protocol type
7. Display the packet details
8. Repeat the process to capture packets continuously.

PROGRAM:

```
import socket
import struct
s. socket. socket (socket. AF_INET, socket. SOCK_RAW,
                   socket. IPPROTO_IP)
s. find ((socket. gethouse by name (socket. gethost
                   by name ()), 0))
s. set sockopt (socket. IPPROTO_IP, socket. IP_
                   HDRINCL, 1)
```

print (" Packet Sniff
try:
while True:
packet, addr
ip-header =
iph = struct
src = socke
dst = sock
protocol = ip
protocols =
name = p
print (
print (
print (
print
except keyboa
print
s. clos

OUTPUT

INPUT
Star
Gene

OUTPUT
Packe
Sou
Des
Pro

```
print("Packet Sniffer started...")
try:
    while True:
        packet, addr = s.recvfrom(65535)
        ip_header = packet[:20]
        iph = struct.unpack('!BBHHHBBH4s4s', ip_header)
        src = socket.inet_ntoa(iph[8])
        dst = socket.inet_ntoa(iph[9])
        protocol = iph[6]
        protocols = {1: "ICMP", 6: "TCP", 17: "UDP"}
        name = protocols.get(protocol, "other")
        print("Source IP:", src)
        print("Destination IP:", dst)
        print("Protocol:", name)
        print("-" * 30)
except KeyboardInterrupt:
    print("Sniffing stopped")
    s.close()
```

OUTPUT

INPUT

Start the packet sniffer

Generate network traffic by pinging a host.

OUTPUT

Packet Sniffer Started

Source IP: 192.168.1.5

Destination IP: 8.8.8.8

Protocol: ICMP

Source IP: 8.8.8.8

Destination IP: 192.168.1.5

Protocol: ICMP

Source IP : 192.168.1.5

Destination IP : 192.168.1.10

Protocol : TCP

RESULT:

Thus, packet-sniffing program was implemented to capture network packets and display their IP's successfully.

Ex No: 15

Date : 10/10/25

AIM:

To analyze ... using the ...

PROCEDURE

1. Run the ...
2. Input ... the ...
3. Press ...
4. Wireshark ... statistica...
5. View ...
6. Wireshark ...
7. It ... infor...
8. The ... graph...
9. Op...
10. ... req...

Ex No: 15 WEB LOG Analysis Using Webalizer
Date: 10/10/24

AIM:

To analyze different types of web logs using the webalizer tool

PROCEDURE

1. Run the webalizer windows version
2. Input the web log file download from the web
3. Press Run Webalizer
4. Webalizer processes the log file and generates statistical reports
5. View the generated report in web browser.
6. Webalizer reads and processes the log file
7. It analyses the requests and visitor information.
8. The tool generates statistical reports and graphs
9. Open the generated report in a web browser
10. Analyze the number of visits, pages and requests, bandwidth usage

Log files

c:\Users\TCS\Downloads\access_log

Target directory

c:\Users\TCS

OUTPUT:

Monthly statistics

